

PROJECT DOCUMENTATION: 4-BIT ALU DESIGN & VERIFICATION

COMPANY NAME: CODTECH IT SOLUTIONS

INTERN NAME: Kakarlamudi Divya

INTERN ID: CITS998

DOMAIN: Digital VLSI Design / RTL Verification

TASK TITLE: Design and Verification of a Basic 4-bit AL

1. Overview

This project focuses on the design and verification of a basic 4-bit Arithmetic Logic Unit (ALU) using Verilog HDL. The ALU is a fundamental building block of a central processing unit (CPU) that performs arithmetic and bitwise logical operations on integer binary numbers.

2. Block Diagram Description

The 4-bit ALU (alu_4bit) takes two 4-bit data inputs, one 3-bit control input, and generates a 4-bit data output along with two status flags.

Input Ports:

1.a [3:0]: 4-bit input operand A

2..b [3:0]: 4-bit input operand B

3.alu_control [2:0]: 3-bit selection lines to decide the ALU operation.

Output ports:

1.alu_result [3:0]: 4-bit output displaying the operation result

2.carry_out: 1-bit flag indicating an arithmetic carry or borrow

3.zero_flag: 1-bit flag indicating if the output result is zero.

3. Functional Table (Operation Codes)

The ALU supports 8 distinct operations based on the 3-bit alu_control signal

Control Code (alu_control)	Operation type	Operation	Discription
3'b000	Arithmetic	Addition	{carry_out, alu_result} = $a + b$
3'b001	Arithmetic	Subtraction	alu_result = $a - b$ (Carry acts as Borrow)
3'b010	Logical	Bitwise and	alu_result = $a \& b$
3'b011	Logical	Bitwise or	alu_result = $a \mid b$
3'b100	Logical	Bitwise xor	alu_result = $a \wedge b$
3'b101	Logical	Bitwise nor	alu_result = $\sim(a \mid b)$

3'b110	Shifting	Left shift	alu_result = a << 1
--------	----------	------------	------------------------

3'b111	Shifting	Right shift	alu_result = a >> 1
--------	----------	-------------	------------------------

4. RTL Architecture and Code Walkthrough

The Design implementation ensures robust hardware generation through the following practices:

Latch Prevention:

alu_result and carry_out are initialized to default values at the beginning of the always @(*) block to avoid unwanted latches

Combinational Logic:

The module uses a standard case statement sensitive to all input transitions.

Flag generation:

1. Carry Flag:

Calculated directly during addition. For subtraction, it indicates a borrow ($a < b$).

2. Zero Flag:

A separate conditional block checks if `alu_result` equals `4'b0000` and dynamically updates the `zero_flag`.

5. Testbench and Verification Strategy

The testbench (`tb_alu_4bit`) creates a self-contained simulation environment to validate all operations

Verification plan

Waveform Generation:

\$dumpfile and \$dumpvars are used to generate VCD (Value Change Dump) logs for waveform viewing.

Console Monitoring:

\$monitor logs data in real-time whenever inputs or outputs change.

Stimulus Driving:

Comprehensive test cases are injected with a #10 time unit delay to observe stable responses.

Executed Test Cases

1.Addition:

$4 + 5 = 9$ (No Carry). Addition with Carry: $12 + 6 = 18$ (Result = 2, Carry = 1).

2.Subtraction: $10 - 4 = 6$ (No Borrow).

3.Bitwise AND:

Checks masking behavior.

4.Bitwise OR:

Checks logical addition.

5.Bitwise XOR:

Identical inputs generate all zeros, validating the zero_flag.

6.Shift Left: Shifts operand A by one bit positins

Source codes:

```
//  
=====
```

// COMPANY NAME: CODTECH IT SOLUTIONS
// INTERN NAME: Kakarlamudi divya
// INTERN ID: CITS998
// DOMAIN: Digital VLSI Design / RTL Verification
// TASK TITLE: Design and Implementation of a Basic 4-bit ALU

```
//
```



```

=====

module alu_4bit (
    input wire [3:0] a,          // 4-bit input operand A
    input wire [3:0] b,          // 4-bit input operand B
    input wire [2:0] alu_control, // 3-bit control signal to
select operation
    output reg [3:0] alu_result, // 4-bit output result of the
operation
    output reg      carry_out,   // Carry out flag for addition
    output reg      zero_flag    // Zero flag set high if
alu_result is 0
);

    // Combination logic block sensitive to any change in
inputs
    always @(*) begin
        // Default values to prevent unwanted latches
        alu_result = 4'b0000;
        carry_out  = 1'b0;

        case (alu_control)

```

```
3'b000: begin // Arithmetic Addition
    {carry_out, alu_result} = a + b;
end
```

```
3'b001: begin // Arithmetic Subtraction
    alu_result = a - b;
    carry_out = (a < b) ? 1'b1 : 1'b0; // Borrow
indicator
end
```

```
3'b010: begin // Bitwise AND
    alu_result = a & b;
end
```

```
3'b011: begin // Bitwise OR
    alu_result = a | b;
end
```

```
3'b100: begin // Bitwise XOR
    alu_result = a ^ b;
end
```

```
3'b101: begin // Bitwise NOR
    alu_result = ~(a | b);
end
```

```

3'b110: begin // Logical Left Shift
    alu_result = a << 1;
end

3'b111: begin // Logical Right Shift
    alu_result = a >> 1;
end

default: begin // Default fallback
    alu_result = 4'b0000;
    carry_out = 1'b0;
end
endcase

// Sequential evaluation for the Zero Flag
if (alu_result == 4'b0000) begin
    zero_flag = 1'b1;
end else begin
    zero_flag = 1'b0;
end
end

endmodule

```

Source code:

```
//  
=====
```

// COMPANY NAME: CODTECH IT SOLUTIONS
// INTERN NAME: kakarlamudi Divya
// INTERN ID: CITS908
// DOMAIN: Digital VLSI Design / RTL Verification
//TASK TITLE: Testbench for Basic 4-bit ALU

```
//  
=====
```

Test bench code:

```
`timescale 1ns / 1ps  
  
module tb_alu_4bit;  
  
    // Inputs to the Device Under Test (DUT)  
    reg [3:0] a;  
    reg [3:0] b;  
    reg [2:0] alu_control;  
  
    // Outputs from the Device Under Test (DUT)
```

```

wire [3:0] alu_result;
wire      carry_out;
wire      zero_flag;

// Instantiate the 4-bit ALU module
alu_4bit uut (
    .a(a),
    .b(b),
    .alu_control(alu_control),
    .alu_result(alu_result),
    .carry_out(carry_out),
    .zero_flag(zero_flag)
);

// Stimulus block
initial begin $dumpfile("dump.vcd"); $dumpvars(0,
tb_alu_4bit); a=4'b0000; b=4'b0000; alu_control=3'b000;
    // Initialize inputs
    a = 4'b0000;
    b = 4'b0000;
    alu_control = 3'b000;

    // Monitor window for console logging
    $monitor("Time=%0dns | Control=%b | A=%b B=%b |
Result=%b | Carry=%b | Zero=%b",

```

```
    $time, alu_control, a, b, alu_result, carry_out,  
zero_flag);
```

```
#10;
```

```
// Test Case 1: Addition (4 + 5 = 9)
```

```
a = 4'b0100; b = 4'b0101; alu_control = 3'b000; #10;
```

```
// Test Case 2: Addition with Carry (12 + 6 = 18 ->  
Result 2, Carry 1)
```

```
a = 4'b1100; b = 4'b0110; alu_control = 3'b000; #10;
```

```
// Test Case 3: Subtraction (10 - 4 = 6)
```

```
a = 4'b1010; b = 4'b0100; alu_control = 3'b001; #10;
```

```
// Test Case 4: Bitwise AND
```

```
a = 4'b1100; b = 4'b1010; alu_control = 3'b010; #10;
```

```
// Test Case 5: Bitwise OR
```

```
a = 4'b1100; b = 4'b1010; alu_control = 3'b011; #10;
```

```
// Test Case 6: Bitwise XOR (Should trigger zero flag if  
identical)
```

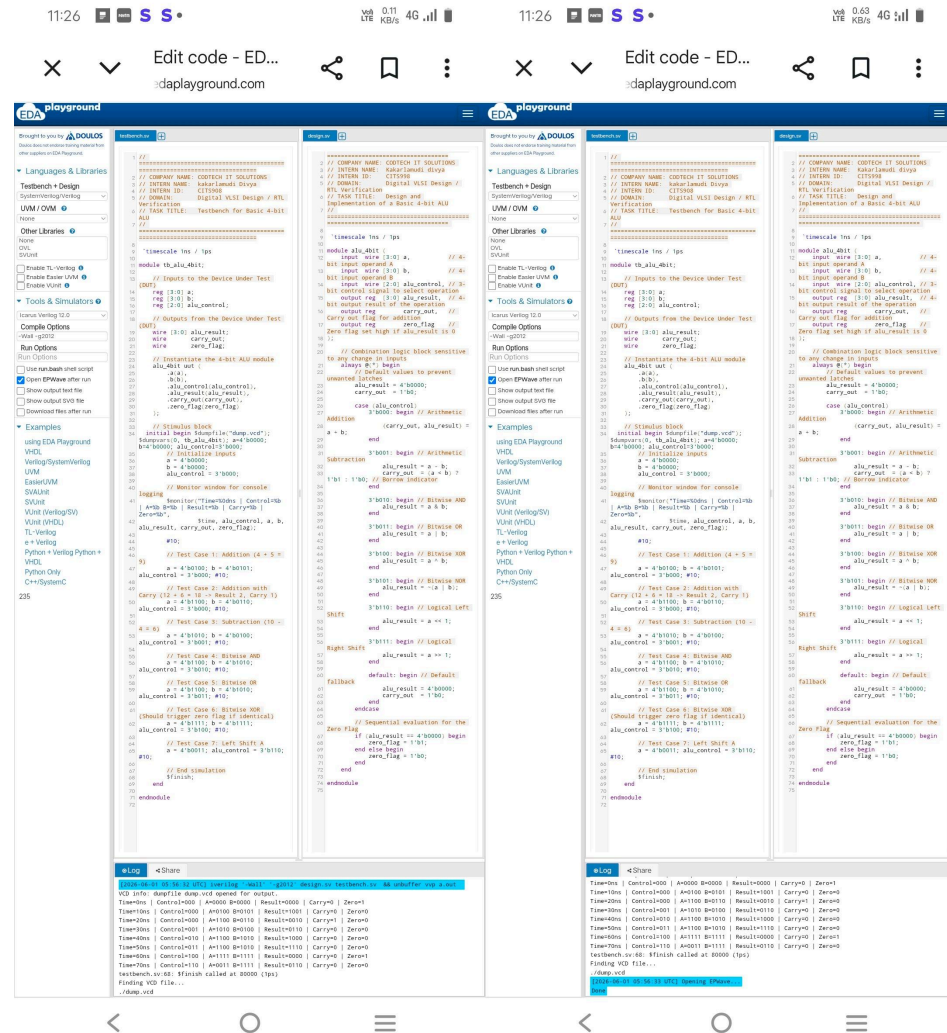
```
a = 4'b1111; b = 4'b1111; alu_control = 3'b100; #10;
```

```
// Test Case 7: Left Shift A
a = 4'b0011; alu_control = 3'b110; #10;

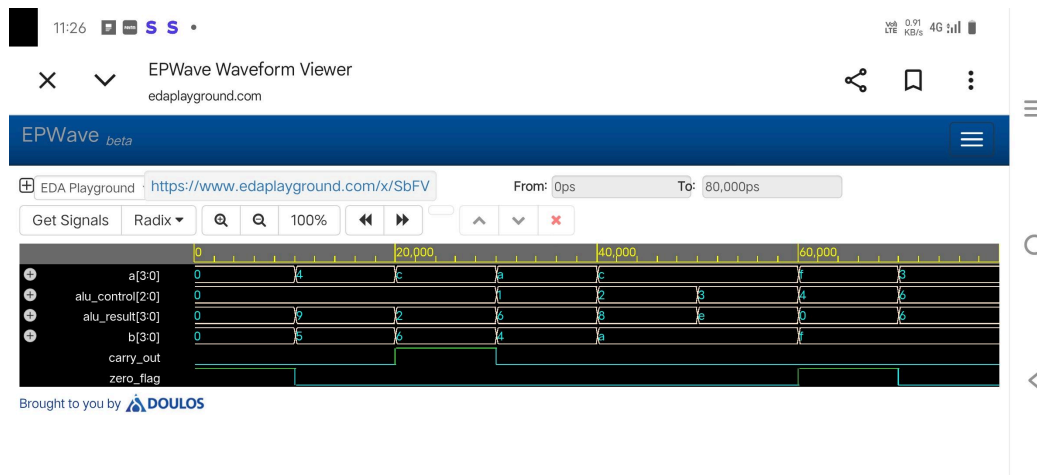
// End simulation
$finish;
end

endmodule
```

Simulation results & outputs



Timing waveform (epwave window)



Conclusion:

The 4-bit ALU design was successfully implemented and verified using Verilog HDL. The simulation logs and waveforms confirm that both the arithmetic-logic hardware and the conditional status flags (zero_flag, carry_out) function exactly as specified in the architecture requirements.

